ASSIGNMENT – 12.3

HT.NO: 2303A52493

BATCH: 50

TASK – 01:

Question:

Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

Tasks

1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).

2. Implement the following sorting algorithms using AI assistance:

o Quick Sort

o Merge Sort

3. Measure and compare runtime performance for large datasets.

4. Write a function to display the top 10 students based on CGPA.

Prompt:

Generate a python code for sorting student records for placements drive in SR University.Sort students by CGPA in decending order.Implement the sorting methods like Quick sort and Merge sort.Store the result in the format of Name,Roll no,CGPA.

Code:

```
import random

import time

def make_student(i):

    return {

        "name": f"Student{i}",
```

```
        "roll": f"R{i:05d}",
        "cgpa": round(random.uniform(5.0, 10.0), 3)  }
def gen_students(n):
    return [make_student(i) for i in range(1, n + 1)]
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    out = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i]["cgpa"] >= right[j]["cgpa"]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    out.extend(left[i:]); out.extend(right[j:])
    return out
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]["cgpa"]
    low = [x for x in arr if x["cgpa"] > pivot]
    equal = [x for x in arr if x["cgpa"] == pivot]
    high = [x for x in arr if x["cgpa"] < pivot]
    return quick_sort(low) + equal + quick_sort(high)
def show_top(students, k=10):
    print(f"Top {k} students by CGPA:")
```

```
    for i, s in enumerate(students[:k], 1):
        print(f"{i:2d}. {s['name']} {s['roll']} {s['cgpa']:.3f}")
if __name__ == "__main__":
    data = gen_students(50000)  # large dataset
    d1 = list(data); d2 = list(data)
    t0 = time.perf_counter()
    r1 = quick_sort(d1)
    q_time = time.perf_counter() - t0
    t0 = time.perf_counter()
    r2 = merge_sort(d2)
    m_time = time.perf_counter() - t0
    print(f"QuickSort: {q_time:.4f}s | MergeSort: {m_time:.4f}s")
    show_top(r1, 10)
```

Output:

```
AB_11_5.py'
QuickSort: 0.5244s | MergeSort: 0.8960s
Top 10 students by CGPA:
 1. Student4340 R04340 10.000
 2. Student33542 R33542 10.000
 3. Student37241 R37241 10.000
 4. Student38556 R38556 10.000
 5. Student40866 R40866 10.000
 6. Student993 R00993 9.999
 7. Student6801 R06801 9.999
 8. Student8303 R08303 9.999
 9. Student9642 R09642 9.999
10. Student11806 R11806 9.999
PS C:\Users\mouni\OneDrive\Desktop\Ai Ass labs>
```

Explanation:

Here in the output we can see that Students sorted in the form of roll no and by their CGPA using Quick sort and Merge sort.

TASK – 02:

Question:

Implementing Bubble Sort with AI Comments

• Task: Write a Python implementation of Bubble Sort.

• Instructions:

• Students implement Bubble Sort normally.

• Ask AI to generate inline comments explaining key logic (like

swapping, passes, and termination).

• Request AI to provide time complexity analysis.

Prompt:

Generate a python code for Implenemting bubble sort and also use key logic like Swapping,passes and termination.

Code:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
if __name__ == "__main__":
    data = [64, 34, 25, 12, 22, 11, 90]
    print("Before:", data)
    print("After :", bubble_sort(data))
```

Output:

```
PS C:\Users\mouni\OneDrive\Desktop\Ai Ass labs>  c:; cd 'c:\Users\mouni\OneDrive\Deskt
rs\mouni\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\mouni\.vscode\e
y-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '51843' '--' 'C:\Users\mouni\OneD
AB_11_5.py'
Before: [64, 34, 25, 12, 22, 11, 90]
After : [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\mouni\OneDrive\Desktop\Ai Ass labs>
```

Explanation:

we can see that the elements are arrange in ascending order by using bubble sort with key logics of swapping, pass and termination.

TASK – 03:

Question:

Quick Sort and Merge Sort Comparison

• Task: Implement Quick Sort and Merge Sort using recursion.

• Instructions:

• Provide AI with partially completed functions for recursion.

• Ask AI to complete the missing logic and add docstrings.

• Compare both algorithms on random, sorted, and reverse-sorted

lists.

Prompt:

Generate apython code for Quick sort and Merege sort comparison using practically completed functions for recursion, to complete the missing logic and add docstrings. Compare both algorithms on random, sorted, and reverse-sorted.

Code:

```
import random

import time

def quick_sort(arr):

    """

    Recursive Quick Sort (descending order by default).

    Best/Average: O(n log n), Worst: O(n^2) (bad pivot choices, e.g., sorted data).

    Space: O(log n) expected recursion.

    """

    if len(arr) <= 1:

        return arr

    pivot = arr[len(arr) // 2]

    left = [x for x in arr if x > pivot]
```

```
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x < pivot]
    return quick_sort(left) + mid + quick_sort(right)


def merge_sort(arr):
    """
    Recursive Merge Sort (descending order).
    Best/Average/Worst: O(n log n). Space: O(n) extra.
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    out = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] >= right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    out.extend(left[i:]); out.extend(right[j:])
    return out
def timeit(func, data):
    t0 = time.perf_counter()
    func(list(data))
    return time.perf_counter() - t0
def test_case(name, data):
    print(f"\n{name} (n={len(data)})")
```

```
    print(" Quick Sort :", timeit(quick_sort, data))
    print(" Merge Sort :", timeit(merge_sort, data))
if __name__ == "__main__":
    n = 30000
    random_data = [random.random() for _ in range(n)]
    sorted_data = sorted(random_data, reverse=True)
    reverse_data = list(reversed(sorted_data))
    test_case("Random", random_data)
    test_case("Sorted desc", sorted_data)
    test_case("Reverse sorted asc", reverse_data)
    print("\nComplexity summary:")
    print("Quick  : Best O(n log n), Avg O(n log n), Worst O(n^2)")
    print("Merge  : Best/Avg/Worst O(n log n)")
```

Output:

```
PS C:\Users\mouni\OneDrive\Desktop\Ai Ass labs>  c:; cd 'c:\Users\mouni\OneDrive\Desktop\Ai Ass labs'; & '
rs\mouni\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\mouni\.vscode\extensions\ms-python.
y-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61899' '--' 'C:\Users\mouni\OneDrive\Desktop\Ai Ass
AB_11_5.py'
 Merge Sort : 0.29736520000733435

Sorted desc (n=30000)
 Quick Sort : 0.14565500000026077
 Merge Sort : 0.17066169998724945

Reverse sorted asc (n=30000)
 Quick Sort : 0.1529797000112012
 Merge Sort : 0.18826689998968504

Complexity summary:
Quick  : Best O(n log n), Avg O(n log n), Worst O(n^2)
Merge  : Best/Avg/Worst O(n log n)
PS C:\Users\mouni\OneDrive\Desktop\Ai Ass labs>
```

Explanation:

In this code we can see the comparisons of quick sort and merge sort by using recursion and also complexities are mentioned.

TASK – 04:

Question:

(Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products,

each with attributes like product ID, name, price, and stock quantity. Store staff

need to:

1. Quickly search for a product by ID or name.

2. Sort products by price or quantity for stock analysis.

Task:

• Use AI to suggest the most efficient search and sort algorithms for this

use case.

• Implement the recommended algorithms in Python.

• Justify the choice based on dataset size, update frequency, and

performance requirements.

Prompt:

Generate a python code for A retail store's inventory system contains thousands of products,each with attributes like product ID, name, price, and stock quantity. Store staff.

Code:

```
products = [
    {"id":"P1","name":"Widget","price":19.99,"qty":120},
    {"id":"P2","name":"Gadget","price":9.99,"qty":350},
    {"id":"P3","name":"Widget","price":18.50,"qty":80},]
print(f"{'ID':<5}{'NAME':<10}{'PRICE':<8}{'QTY':<5}")
for p in products:
    print(f"{p['id']:<5}{p['name']:<10}{p['price']:<8.2f}{p['qty']:<5}")
by_id = {p["id"]: p for p in products}
by_name = {}
for p in products:
    by_name.setdefault(p["name"].lower(), []).append(p)
print("\nSearch ID P2:", by_id.get("P2"))
print("Search name widget:", by_name.get("widget"))
sorted_by_price = sorted(products, key=lambda p: p["price"])
```

```
sorted_by_qty_desc = sorted(products, key=lambda p: p["qty"], reverse=True)
print("\nSorted by price (asc):")
for p in sorted_by_price:
    print(f"{p['id']} {p['price']:.2f}")
print("\nSorted by qty (desc):")
for p in sorted_by_qty_desc:
    print(f"{p['id']} {p['qty']}")
```

Output:

```
PS C:\Users\mouni\OneDrive\Desktop\Ai Ass labs>  c:; cd 'c:\Users\mouni\OneDrive\Desktop\Ai Ass labs'; & 'c:\Use
rs\mouni\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\mouni\.vscode\extensions\ms-python.debugp
y-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '55866' '--' 'C:\Users\mouni\OneDrive\Desktop\Ai Ass labs\L
AB_11_5.py'
ID   NAME      PRICE   QTY
P1   Widget    19.99   120
P2   Gadget    9.99    350
P3   Widget    18.50   80

Search ID P2: {'id': 'P2', 'name': 'Gadget', 'price': 9.99, 'qty': 350}
Search name widget: [{'id': 'P1', 'name': 'Widget', 'price': 19.99, 'qty': 120}, {'id': 'P3', 'name': 'Widget',
'price': 18.5, 'qty': 80}]

Sorted by price (asc):
P2 9.99
P3 18.50
P1 19.99

Sorted by qty (desc):
P2 350
P1 120
P3 80
PS C:\Users\mouni\OneDrive\Desktop\Ai Ass labs>
```

Explanation:

We have generated a code for a retail store's inventory system contains thousands of products,each with attributes like product ID, name, price, and stock quantity. Store staff.

TASK – 05:

Question:

Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing

stock price movements. The requirement is to quickly sort stocks by daily

gain/loss and search for specific stock symbols efficiently.

• Use GitHub Copilot to fetch or simulate stock price data (Stock

Symbol, Opening Price, Closing Price).

• Implement sorting algorithms to rank stocks by percentage change.

• Implement a search function that retrieves stock data instantly when a

stock symbol is entered.

• Optimize sorting with Heap Sort and searching with Hash Maps.

• Compare performance with standard library functions (sorted(), dict

lookups) and analyze trade-offs.

Prompt:

Generate a python code for An AI-powered FinTech Lab at SR University is building a tool for analyzingstock price movements. The requirement is to quickly sort stocks by dailygain/loss and search for specific stock symbols efficiently.

• Use GitHub Copilot to fetch or simulate stock price data (Stock,Symbol, Opening Price, Closing Price).

Code:

```
import random

stocks = []

for i in range(20):

    sym = f"S{i:03}"

    openp = round(random.uniform(10, 200), 2)

    closep = round(openp * random.uniform(0.9, 1.1), 2)

    change = round((closep - openp) / openp * 100, 2)

    stocks.append({"sym": sym, "open": openp, "close": closep, "change": change})

by_sym = {s["sym"]: s for s in stocks}

top5 = sorted(stocks, key=lambda x: x["change"], reverse=True)[:5]

print("Top 5 gainers:", [(x["sym"], x["change"]) for x in top5])

print("Search S005:", by_sym.get("S005"))
```

Output:

```
PS C:\Users\mouni\OneDrive\Desktop\Ai Ass labs>  c:; cd 'c:\Users\mouni\OneDrive\Desktop\Ai Ass labs' ; & 'c:
rs\mouni\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\mouni\.vscode\extensions\ms-python.de
y-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '64853' '--' 'C:\Users\mouni\OneDrive\Desktop\Ai Ass la
AB_11_5.py'
Top 5 gainers: [('S016', 8.5), ('S008', 8.16), ('S014', 7.79), ('S001', 7.31), ('S017', 5.65)]
Search S005: {'sym': 'S005', 'open': 89.34, 'close': 89.51, 'change': 0.19}
PS C:\Users\mouni\OneDrive\Desktop\Ai Ass labs>
```

Explanation:

Stock sorting/search code: sorts by % change and dict lookup is instant (O(1)), so it's fast for real-time analytics.